

COMPANION
to
ChatGPT Teaches TikZ
Chapter 2: Styling (B)

Contents

2.1	What This Companion Adds	2
2.2	Reading a Node Command	3
2.3	Labels Around a Point	4
2.4	Named Coordinates	5
2.5	Named Nodes	6
2.6	Reusable Styles	6
2.7	Worked Project: A Four-Stage Process	7
2.8	Debugging Workshop	9
2.9	Exercises	10
2.10	Solutions	10
2.11	ChatGPT Workshop	12
2.12	Final Checklist	12

2.1 What This Companion Adds

Chapter 2 introduced nodes, labels, named coordinates, named nodes, and simple styles. These ideas turn isolated coordinates into objects that can be read, named, reused, and revised.

This Companion develops four habits:

- separate a node’s internal name from its visible text;
- give important points and objects meaningful names;
- use placement options for labels, not for changing coordinates; and
- replace genuinely repeated options by one descriptive style.

Design

The visual design remains plain. Color, rounded corners, and advanced positioning belong to later chapters. Here the goal is to understand the structure of a node and the purpose of a name.

2.2 Reading a Node Command

Consider

```
\node[draw] (start) at (1,2) {Begin};
```

The command contains six pieces of information.

1. `\node` creates a node.
2. `[draw]` places a border around it.
3. `(start)` gives it an internal name.
4. `at` introduces its position.
5. `(1,2)` supplies the coordinate.
6. `{Begin}` supplies the visible text.

The semicolon ends the command.

2.2.1 Name and Visible Text

The reader and TikZ use different identifiers:

```
\node[draw] (first) at (0,0) {Read the problem};
```

The reader sees **Read the problem**; TikZ knows the object as **first**. The name can remain unchanged even if the visible wording is revised.

Choose names for the source, not for the printed page. A name such as `input`, `proof`, or `finish` is easier to remember than `n1` when the picture is revised later.

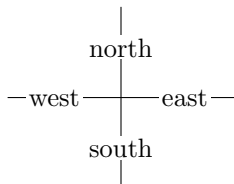
2.3 Labels Around a Point

The options `above`, `below`, `left`, and `right` place text relative to a coordinate.

```
\begin{tikzpicture}
\draw (-1.5,0) -- (1.5,0);
\draw (0,-1.2) -- (0,1.2);

\node[above=5mm,fill=white,inner sep=1pt] at (0,0) {north};
\node[below=5mm,fill=white,inner sep=1pt] at (0,0) {south};
\node[left=5mm,fill=white,inner sep=1pt] at (0,0) {west};
\node[right=5mm,fill=white,inner sep=1pt] at (0,0) {east};
\end{tikzpicture}
```

See



The coordinate remains $(0,0)$ in all four commands. Only the position of the text changes. The explicit distance separates the labels from the coordinate, and the white node background keeps the crossing lines from running through the words.

Common Mistake

Do not write `right=of` here. The option `right` places a label beside one point. Relative positioning with `right=of` is introduced in Chapter 6.

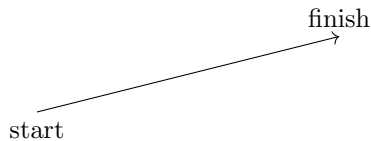
2.4 Named Coordinates

A named coordinate stores a position without printing anything.

```
\begin{tikzpicture}
  \coordinate (start) at (0,0);
  \coordinate (finish) at (4,1);

  \draw[->] (start) -- (finish);
  \node[below] at (start) {start};
  \node[above] at (finish) {finish};
\end{tikzpicture}
```

See



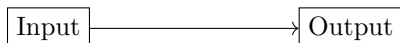
If the coordinate of `finish` changes, both the arrow and its label follow automatically. This is the first important advantage of naming objects.

2.5 Named Nodes

A named node combines position, visible text, and an object that can later be used as a connection endpoint.

```
\begin{tikzpicture}
  \node[draw] (input) at (0,0) {Input};
  \node[draw] (output) at (4,0) {Output};
  \draw[->] (input) -- (output);
\end{tikzpicture}
```

See



TikZ connects the borders of the nodes. The arrow does not run through the visible text.

2.6 Reusable Styles

When several objects share the same options, give those options a meaningful name.

```
\tikzset{
  process/.style={draw}
}

\node[process] (read) at (0,0) {Read};
\node[process] (think) at (2.5,0) {Think};
\node[process] (write) at (5,0) {Write};
```

The style says what role the objects have. Its definition says how that role is displayed. If the border should later change, edit the style once.

2.6.1 When a Style Helps

A style is useful when an option collection is repeated or when the name adds meaning. It is unnecessary for a single ordinary object.

Avoid very general names that TikZ may already use. For example, `step` is already a TikZ key. A descriptive name such as `process node` is safer.

2.7 Worked Project: A Four-Stage Process

The project is built in four stages so that structure and formatting remain separate.

2.7.1 Stage 1: Place and Name the Nodes

```
\node (read) at (0,0) {Read};  
\node (think) at (2.3,0) {Think};  
\node (draw) at (4.6,0) {Draw};  
\node (check) at (6.9,0) {Check};
```

2.7.2 Stage 2: Add One Style

```
\tikzset{process node/.style={draw}}
```

Apply `process node` to all four nodes.

2.7.3 Stage 3: Connect the Names

```

\draw[->] (read) -- (think);
\draw[->] (think) -- (draw);
\draw[->] (draw) -- (check);

```

2.7.4 The Complete Picture

```

\begin{tikzpicture}
  \tikzset{process node/.style={draw}}

  \node[process node] (read) at (0,0) {Read};
  \node[process node] (think) at (2.3,0) {Think};
  \node[process node] (draw) at (4.6,0) {Draw};
  \node[process node] (check) at (6.9,0) {Check};

  \draw[->] (read) -- (think);
  \draw[->] (think) -- (draw);
  \draw[->] (draw) -- (check);
\end{tikzpicture}

```

See



Design

The source uses names for structure and one style for appearance. Changing a visible word does not require changing an arrow; changing the shared border does not require editing four nodes.

2.8 Debugging Workshop

2.8.1 Confusing a Name with Visible Text

This node is named `start`, but prints `Begin`:

```
\node[draw] (start) at (0,0) {Begin};
```

Writing `(Begin)` in a later connection is therefore incorrect.

2.8.2 Using a Name Before Defining It

Define a coordinate or node before a command refers to it. TikZ cannot connect to an object that does not yet exist.

2.8.3 Forgetting Braces Around Text

Incorrect:

```
\node at (0,0) Start;
```

Correct:

```
\node at (0,0) {Start};
```

2.8.4 Missing Commas in an Option List

Separate several options with commas. The following is correct:

```
\node[draw,above] at (0,0) {Start};
```

2.9 Exercises

1. Place the word **Center** at (0,0).
2. Place four labels around the same coordinate.
3. Define named coordinates **leftpoint** and **rightpoint**, then draw an arrow between them.
4. Create three named bordered nodes in a horizontal row.
5. Change the visible text of one node without changing its internal name.
6. Define a style called **stage node** containing only **draw**.
7. Apply **stage node** to four named nodes.
8. Build a four-stage process diagram and then move one node by editing only its coordinate.

2.10 Solutions

2.10.1 Exercises 1–4

```
\node at (0,0) {Center};
```

```
\node[above] at (0,0) {Top};  
\node[below] at (0,0) {Bottom};  
\node[left] at (0,0) {Left};  
\node[right] at (0,0) {Right};
```

```
\coordinate (leftpoint) at (0,0);  
\coordinate (rightpoint) at (4,1);  
\draw[->] (leftpoint) -- (rightpoint);
```

```
\node[draw] (A) at (0,0) {One};  
\node[draw] (B) at (2.5,0) {Two};  
\node[draw] (C) at (5,0) {Three};
```

2.10.2 Exercises 5–7

```
\node[draw] (input) at (0,0) {New wording};
```

```
\tikzset{stage node/.style={draw}}
```

```
\node[stage node] (A) at (0,0) {A};  
\node[stage node] (B) at (2,0) {B};  
\node[stage node] (C) at (4,0) {C};  
\node[stage node] (D) at (6,0) {D};
```

Exercise 8 has many correct solutions. Check that the arrows still use node names after one coordinate changes.

2.11 ChatGPT Workshop

Use prompts that distinguish structure, names, visible text, and appearance.

1. Explain every part of `\node[draw] (start) at (0,0) {Begin};`.
2. Compare a coordinate name, a node name, and visible node text.
3. Check my Chapter 2 picture for unclear or undefined names. Do not use syntax from later chapters.
4. Turn the repeated `draw` options in my nodes into one style named `process node` without changing the picture.
5. Change the visible labels in this diagram without changing its internal node names or connections.

2.12 Final Checklist

Before continuing to Chapter 3, check that you can

1. distinguish a node's internal name from its visible text;
2. place labels above, below, left, and right of a point;
3. define and reuse a named coordinate;
4. create a named bordered node;
5. connect two objects by their names; and
6. define and apply one simple semantic style.